PES-VCS LAB REPORT

NAME: DHARANI S

SRN: PES2UG24CS157

REPOSITORY

PHASE 1: OBJECT STORAGE FOUNDATION

Filesystem Concepts: Content-addressable storage, directory sharding, atomic writes, hashing for integrity

Implementation Summary

`object_write` builds a full object by prepending a type header (`"blob <size>\0"`) to the data, computes a SHA-256 hash of the entire object, checks for deduplication, creates the shard directory (first 2 hex chars), writes to a temp file, calls `fsync()`, then atomically renames it to the final path. The shard directory is also `fsync()`-ed to persist the rename.

`object_read` builds the file path from the hash, reads the entire file into memory, recomputes the SHA-256 and compares it against the expected hash for integrity verification, parses the type and size from the header, then returns a heap-allocated copy of the data portion after the null byte.

Screenshot 1A – `./test_objects` output

```
./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bb
e9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea0922
7ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B – Sharded object store

```
find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda47
1e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e1
1ec4f476
```

PHASE 2: TREE OBJECTS

Filesystem Concepts: Directory representation, recursive structures, file modes and permissions

Implementation Summary

`tree_from_index` heap-allocates an `Index`, loads it, sorts entries by path, then calls a recursive helper `write_tree_recursive`. The helper walks the sorted entries: if a path has no `/` relative to the current prefix, it's a direct file entry; if it does, all entries sharing that subdirectory prefix are grouped and the helper recurses to produce a subtree object. Each level serializes its `Tree` struct and writes it to the object store via `object_write`, returning the root tree hash.

Screenshot 2A – `./test_tree` output

```
make test_tree && ./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B – Raw tree object (xxd)

```
~/OS  main
xxd .pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
00000000: 626c 6f62 2031 3200 6865 6c6c 6f20 776f  blob 12.hello wo
00000010: 726c 640a                                rld.
```

PHASE 3: THE INDEX (STAGING AREA)

Filesystem Concepts: File format design, atomic writes, change detection using metadata

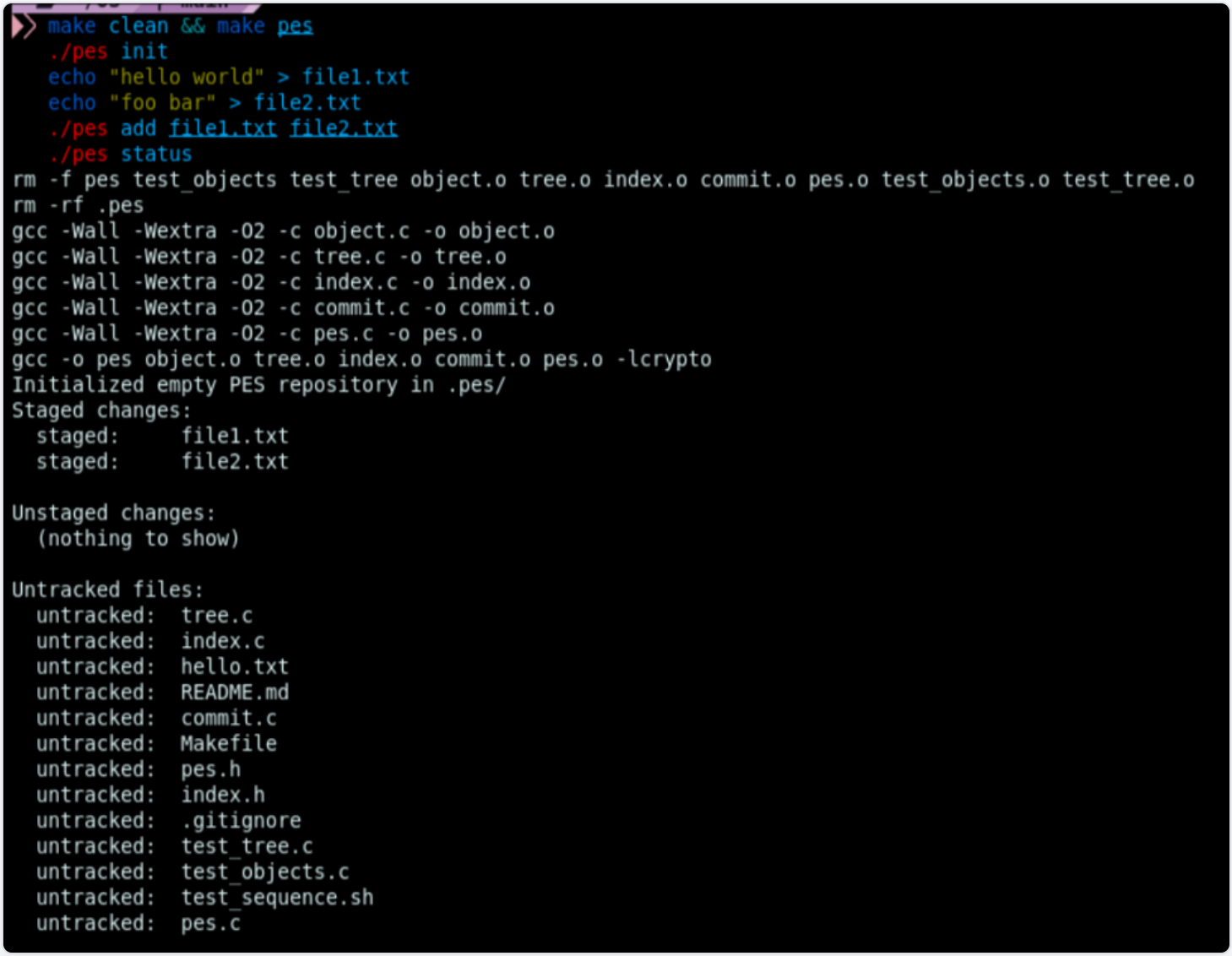
Implementation Summary

`index_load` opens `.pes/index` and parses each line with `sscanf` using the format `%o %64s %SCNu64 %SCNu64 %511s` into mode, hex hash, mtime, size, and path fields. If the file does not exist, it returns 0 with an empty index – not an error.

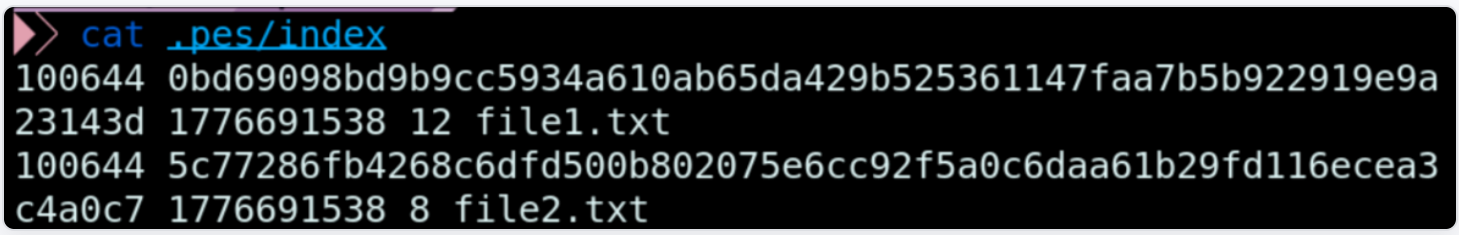
`index_save` heap-allocates a sorted copy of the index, sorts entries by path using `qsort`, writes each entry to a temp file using `fprintf` with `PRIu64`/`PRIu32` format specifiers, calls `fflush` + `fsync` + `fclose`, then atomically renames the temp file over the real index file.

`index_add` opens the target file, reads its contents, calls `object_write` to store a blob, calls `lstat` to get metadata, then either updates an existing index entry (found via `index_find`) or appends a new one, finally calling `index_save`.

Screenshot 3A — `pes init` → `pes add` → `pes status`



Screenshot 3B — `cat .pes/index`



PHASE 4: COMMITS AND HISTORY

Filesystem Concepts: Linked structures on disk, reference files, atomic pointer updates

Implementation Summary

`commit_create` performs the following steps:

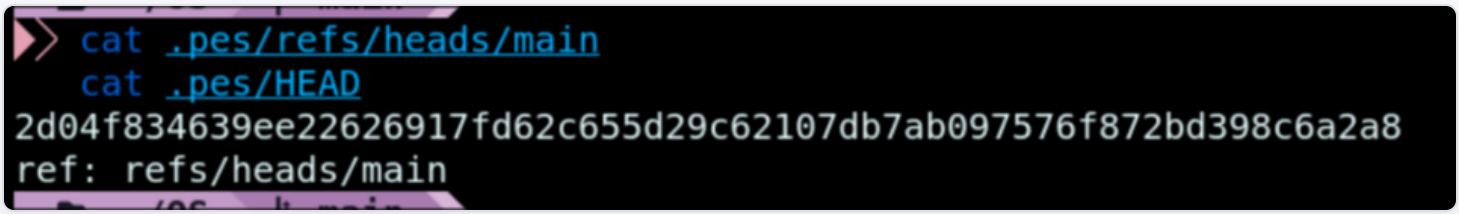
1. Calls `tree_from_index` to build and store the directory tree, getting the root tree hash
2. Calls `head_read` to get the parent commit hash — if it fails (first commit), `has_parent` is set to 0
3. Fills the `Commit` struct with the tree hash, optional parent, author string from `pes_author()`, and current Unix timestamp from `time(NULL)`
4. Calls `commit_serialize` to produce the text-format commit buffer
5. Calls `object_write` with `OBJ_COMMIT` to store the commit object
6. Calls `head_update` to atomically move the branch ref to the new commit hash

Screenshot 4A – `pes log` with three commits

```
./pes log
commit 2d04f834639ee22626917fd62c655d29c62107db7ab097576f872bd398c6a2a8
Author: Dharani_S PES2UG24CS157
Date:   1776691887

    Add farewell

commit 6df58f0fe76eb381466e0ce70535591897ac9cf4923c2c3121a1e044357db761
Author: Dharani_S PES2UG24CS157
Date:   1776691857

    Add world

commit e145651525f38b6a888831493b49801d2ee104cc3ede342c6c82f0c1930b2eca
Author: Dharani_S PES2UG24CS157
Date:   1776691829

    Initial commit

commit 8ca64ecadea2f334c616daf656ca41e7d7008b9a8ee8788a1bce7dbce53f96c6
Author: Dharani_S PES2UG24CS157
Date:   1776691700

    Add farewell

commit 0c7fe9d0f11f93f4c21f23357c4b13f6da27f2aba5d0c00cd799156cab7e1137
Author: Dharani_S PES2UG24CS157
Date:   1776691700

    Add world

commit 3e469f024c21b1e04ff8c40544286b5013b19bada93aa1ae9b0cff72a5aee32b
Author: Dharani_S PES2UG24CS157
Date:   1776691700

    Initial commit
```

Screenshot 4B – `find .pes -type f | sort`

Screenshot 4C – Reference chain



Final – Integration test

```
make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 2c67749a25fe... Initial commit

Log after first commit:
commit 2c67749a25fe957b17c19bc29f282b6e1917bfe2342d26900ba6d5c9087fecd4
```

```
Author: Dharani_S PES2UG24CS157
Date:   1776692133

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
0f98e17b7b7dd18942af84bee58859939c897a93f59a11634417c5628d85bb63

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/0f/98e17b7b7dd18942af84bee58859939c897a93f59a11634417c5628d85bb63
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/2c/67749a25fe957b17c19bc29f282b6e1917bfe2342d26900ba6d5c9087fecd4
.pes/objects/32/0dbeb0fa4ef164810e0609b8a13f8a0836f24336289390c88ef1c1b9467d33
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

PHASE 5: BRANCHING AND CHECKOUT (ANALYSIS)

Q5.1 – Implementing `pes checkout <branch>`

To implement `pes checkout <branch>`, two things must happen in `.pes/`:

1. `HEAD` must be updated to contain `ref: refs/heads/<branch>`
2. If the branch does not exist yet (creating a new branch), a new file must be created at `.pes/refs/heads/<branch>` containing the current commit hash

The working directory must then be updated to match the target branch's tree. This means reading the target commit object, getting its tree hash, recursively walking the tree, and writing each blob's contents back to the corresponding file path – creating subdirectories as needed. Files that

exist in the current tree but not the target must be deleted. Files only in the target must be created. Files that differ must be overwritten.

The complexity comes from several sources. First, the working directory update must be atomic enough that a failure midway does not leave the repository in a broken half-switched state. Second, untracked files must not be touched — only tracked files should be updated or removed. Third, nested directory structures require recursive tree walking in both the old and new trees. Fourth, file permissions (executable bit) must be restored correctly from the tree entry modes.

Q5.2 — Detecting dirty working directory conflicts

For each file that differs between the source and target branch trees, the algorithm is:

1. Look up the file's entry in the current index
2. Call `stat()` on the actual working directory file
3. Compare `stat.st_mtime` and `stat.st_size` against the index entry's `mtime_sec` and `size` fields
4. If they differ, the file has been modified since it was last staged — this is a dirty file
5. If the file exists in the working directory but has no index entry, it is untracked and can be ignored unless the target branch would create a tracked file at the same path, which is also a conflict

If any file that differs between the two branch trees is dirty according to this check, checkout must refuse with an error message listing the conflicting paths. This approach avoids re-hashing file contents by using metadata as a fast proxy for change detection, which is the same technique used by Git's index.

Q5.3 — Detached HEAD and recovery

In detached HEAD state, `HEAD` contains a raw commit hash instead of a branch reference like `ref: refs/heads/main`. When new commits are made in this state, `head_update` writes the new commit hash directly into `HEAD` rather than updating a branch file. Each new commit correctly points to its parent, forming a valid chain.

However, once the user switches to another branch, `HEAD` is overwritten with the new branch reference. The commits made in detached HEAD state are now unreachable — no branch points to them. They will not appear in `pes log` and will be deleted by garbage collection.

To recover those commits, the user can scan all objects in `.pes/objects/` for commit-type objects, read each one, and identify commits not reachable from any branch ref. Alternatively, if the commit hashes were visible on screen during the session, the user can directly create a new branch pointing at the desired hash:

```
echo "<commit-hash>" > .pes/refs/heads/recovery-branch
```

This makes the commit reachable again and restores the full history chain behind it.

PHASE 6: GARBAGE COLLECTION (ANALYSIS)

Q6.1 — Garbage collection algorithm

The algorithm is a mark-and-sweep over the object graph:

Mark phase:

1. Start with every ref in `.pes/refs/` and the commit pointed to by `HEAD`
2. For each reachable commit: mark it, then mark its tree, then recursively mark all blobs and subtrees reachable from that tree, then follow the parent pointer and repeat
3. Use a hash set (implemented as a sorted array of `ObjectID` structs, or a hash table keyed on the 32-byte hash) to track all reachable object hashes

Sweep phase:

4. Walk every file under `.pes/objects/` using `find` or `opendir`/`readdir`
5. For each object file, parse its hash from the path

6. If the hash is not in the reachable set, delete the file
7. Remove any empty shard directories

Estimate for 100,000 commits, 50 branches:
Assuming an average of 10 files per commit snapshot with heavy sharing between commits (say 3 new blobs per commit on average), the object store contains roughly 100,000 commits + 100,000 trees + 300,000 blobs = ~500,000 objects. The reachability walk visits every reachable object once, so it visits all ~500,000 objects. The sweep also scans all ~500,000 files. Total objects visited: approximately 1,000,000 operations.

Q6.2 — GC race condition with concurrent commit

The race condition proceeds as follows:

1. GC starts and scans all branch refs, building the reachable set. At this moment, a new blob `B` has just been written to the object store by `pes add`, but no commit references it yet.
2. GC does not find `B` in the reachable set since no commit or tree points to it.
3. GC deletes `B` from the object store.
4. `pes commit` runs, calls `tree_from_index`, builds a tree object that references blob `B`'s hash, and writes the tree. The tree object is stored successfully.
5. `pes commit` writes the commit object pointing to the tree, and calls `head_update`.
6. The repository now contains a commit whose tree references a deleted blob — the object store is corrupt.

Git avoids this race condition using a **grace period**: objects newer than a configurable threshold (default 2 weeks, set by `gc.pruneExpire`) are never deleted by GC regardless of reachability. This gives all in-flight operations more than enough time to complete and add references to new objects before they become eligible for collection. Git's GC also writes a `gc.pid` lock file to prevent two GC processes from running simultaneously.

FILE SUMMARY

FILE	DESCRIPTION
`object.c`	Content-addressable object store — `object_write`, `object_read`
`tree.c`	Tree serialization and recursive construction — `tree_from_index`
`index.c`	Staging area — `index_load`, `index_save`, `index_add`
`commit.c`	Commit creation and history walking — `commit_create`